Internet Programming I: Individual Project Guideline

What to build, how it is checked every two weeks, and how to stay in control of your code

Course	Internet Programming I · I4-GIC · ITC · 2026–2027 · Mr. CHUN Thavorac
Type	Individual, one project for the whole semester
Timeline	Week 2 (domain approved) to week 16 (final check)
Stack	Vue 3 + TypeScript, NestJS, PostgreSQL with Prisma
Weight	80% of the course grade, over 7 checks

Your project in one paragraph

Each student builds one individual web application during the semester, from week 2 to week 16, with Vue 3 + TypeScript, NestJS, PostgreSQL with Prisma. You choose the domain, write the spec, and build the app step by step. Every two weeks a score check marks what you have built and whether you can explain it. There is no final presentation: your grade is built over the semester.

Choose your domain

Pick one of these, or propose your own idea of the same size. Your domain must be approved by the teacher at Practice 1 (week 2). Two students may pick the same domain, but their specs and code must be their own.

Domain	What the app does
Library loans	Members borrow and return books; librarians manage the catalogue and overdue loans
Clinic appointments	Patients book time slots; doctors manage their schedule; admins manage doctors
Shop inventory	Staff record stock in and out; managers see low-stock alerts and reports
Club events	Members register for events with a capacity limit; organisers create events
Lab equipment booking	Students reserve equipment; lab staff approve and track returns
Restaurant orders	Waiters take orders per table; kitchen sees a queue; manager edits the menu
Student internship tracker	Students log applications and status; advisors comment and approve
Bus ticket booking	Passengers book seats on trips; operators manage routes and trips
Gym membership	Members book classes; trainers manage classes; admins manage plans
Lost and found	Users report lost or found items; staff match and close cases
Recipe sharing	Users post recipes with ingredients; moderators hide reported posts
Helpdesk tickets	Users open tickets; agents are assigned, reply and close them
Room rental	Tenants see invoices and report issues; landlords manage rooms and payments
Volunteer shifts	Volunteers sign up for shifts; coordinators publish shifts and track attendance
Course feedback	Students rate lectures anonymously; teachers see aggregated results

Minimum requirements by week 16

- ☐ At least 2 user roles with different permissions (for example member and admin)
- ☐ At least 3 related data entities (for example Member, Book, Loan) with a one-to-many relation
- ☐ Full create, read, update, delete on the main entity, with validation on the backend
- ☐ Login with hashed passwords and JWT; protected routes on both frontend and backend
- ☐ Search or filter, and pagination, on at least one list
- ☐ Automated tests: component tests (Vitest) and API tests (Jest + Supertest) for the key flows
- ☐ A README that lets someone else clone, configure and run the app in under 10 minutes

Repository

Create a private GitHub repository named `ip1-<student-id>` from the course template and add the teacher as a collaborator. Keep this structure:

```
ip1-<student-id>/
  README.md          how to run, features, screenshots
  AI_LOG.md          every AI use: asked, kept, rejected, why
  DECISIONS.md       one line per real design choice
  CHECK.md           filled before each score check
  docs/
    SPEC.md          user stories + acceptance criteria
    wireframes/
  frontend/          Vue 3 + Vite + TypeScript
  backend/           NestJS + Prisma
  .github/workflows/ CI (from Check 6)
```

AI_LOG.md: one entry per meaningful AI use

```
## 2026-11-03 · Login form validation
Tool: <assistant name>
Asked: generate a Vue login form with email/password validation
Kept: the form layout and the email regex check
Rejected: it stored the token in localStorage without expiry;
          I kept it in memory + refresh flow (see DECISIONS.md)
Checked by: tested empty, wrong and correct input by hand
```

DECISIONS.md: one line per real choice

```
- 2026-10-20: Prisma instead of TypeORM: typed client, simpler migrations
- 2026-11-10: soft delete for loans, so the history is kept
```

CHECK.md: fill before every score check

```
# Check N
Tag: check-N   Commit: <hash>

## Done (copy the checklist, tick what works)
- [x] ...
- [ ] ... (not done because ...)

## AI use this period (2-3 main AI_LOG entries)

## One decision I made and why
```

```
## What I would show if asked to explain one thing
```

Score checks and milestones

There is no end-of-semester presentation. Your project is scored seven times, at the start of the practice sessions in weeks 4, 6, 8, 10, 12, 14 and 16.

Criterion (each check)	Points	What earns it
Milestone completion	5	The items on that check's checklist work in your tagged commit
Engineering quality	2	Small meaningful commits, tests where required, README, AI_LOG.md and DECISIONS.md kept up
Explain-back	3	Correct answers about your own code, written (everyone) and live (about 10 students per check, chosen at random)

- **Explain-back cap:** below 1.5 of 3 on explain-back halves your milestone points for that check.
- **Late or missed check:** show the work at the next check for at most half of the milestone points.
- **Before each check:** push the git tag `check-N` and fill CHECK.md. Work committed after the tag is not marked for that check.

Component	Weight
Checks 1 to 6 (10% each)	60%
Check 7, final project state with a live change task	20%
Final exam, CLOSED: reading and reasoning about code	20%

Below is the exact checklist for each check. The practice session before each check helps you build it.

Check 1 · Week 4 practice

Prepared in Practice 1.

- ☐ docs/SPEC.md: 6–10 user stories with acceptance criteria, at least 2 roles
- ☐ Wireframes for the main screens in docs/wireframes/
- ☐ frontend/ runs with npm run dev from a fresh clone
- ☐ First static screens built from the spec (no real data yet)
- ☐ AI_LOG.md and DECISIONS.md have real entries
- ☐ Small commits with meaningful messages, tag check-1 pushed

Check 2 · Week 6 practice

Prepared in Practice 2.

- ☐ At least 3 pages reached through Vue Router, including one route with a param (for example /books/:id) and a 404 page
- ☐ One Pinia store per main entity, loaded from mock data (src/mock/ or json-server), no hard-coded lists in components
- ☐ An action changes data on one page, and the change is visible on another page without reload

- ☐ At least one form with v-model and validation messages taken from the acceptance criteria
- ☐ npm run build in frontend/ passes with no type errors
- ☐ docs/REVIEW.md covers every component; AI_LOG.md and DECISIONS.md have entries since check 1
- ☐ Small commits with meaningful messages, tag check-2 pushed, CHECK.md filled

Check 3 · Week 8 practice

Prepared in Practice 3.

- ☐ docs/API.md: every endpoint with method, path, body, success status and error statuses
- ☐ backend/ is a NestJS 11 app that starts with npm run start:dev from a fresh clone
- ☐ One module per main resource with a controller and a service; data kept in memory in the service
- ☐ Create and update endpoints use DTO classes with class-validator, and a global ValidationPipe with whitelist is on
- ☐ Statuses match the contract: 201 on create, 400 for an invalid body, 404 for an unknown id
- ☐ backend/requests.http (or a Bruno or Postman collection) has a request for each endpoint and one invalid case each
- ☐ AI_LOG.md, DECISIONS.md and CHECK.md updated, tag check-3 pushed

Check 4 · Week 10 practice

Prepared in Practice 4.

- ☐ backend/prisma/schema.prisma with all main entities and at least one relation; migrations committed in prisma/migrations
- ☐ Services read and write through Prisma, not in-memory arrays: data survives a backend restart
- ☐ npx prisma db seed creates demo data from a seed script
- ☐ The Vue app calls the NestJS API through src/api/ using VITE_API_URL; json-server is no longer used for the main flows
- ☐ CORS allows the frontend origin; an item created in the UI is still there after a page reload
- ☐ 400 and 404 responses from the API are shown in the UI as readable messages
- ☐ README has setup steps for both apps, .env.example files committed, tag check-4 pushed

Check 5 · Week 12 practice

Prepared in Practice 5.

- ☐ User model with passwordHash (bcrypt) and a role; the seed has at least one user per role
- ☐ POST /auth/register and POST /auth/login work; login returns a JWT signed with JWT_SECRET read from .env
- ☐ Protected endpoints return 401 without a token and 403 with the wrong role, matching the guard table in SPEC.md
- ☐ Vue: login page, token in a Pinia auth store, Authorization header added in api/http.ts, router guard on protected pages
- ☐ No passwordHash in any API response; .env in .gitignore and .env.example complete
- ☐ Small commits, AI_LOG.md entries for the auth work, tag check-5 pushed

Check 6 · Week 14 practice

Prepared in Practice 6.

- ☐ docs/TEST_PLAN.md lists the key flows with one test name per acceptance criterion
- ☐ npm run test:e2e passes from a fresh clone, using a separate test database configured in backend/.env.test
- ☐ E2E tests cover at least 3 key flows, including one 401 or 403 case and one IDOR case (404 for another user's record)
- ☐ npm run test:unit in frontend/ passes with at least 2 component tests of real behaviour
- ☐ One advanced feature (file upload, search or pagination) works on both sides, with its acceptance criteria in SPEC.md and its limits tested
- ☐ docs/SECURITY_CHECKLIST.md completed for your project; tag check-6 pushed

Check 7 · Week 16 practice

Prepared in Practice 7.

- ☐ Every story in SPEC.md marked Done or Not done; every Done story works in the tagged commit
- ☐ README.md: purpose, setup from a fresh clone, environment variables, seeded accounts, test commands, project map
- ☐ npm run test:e2e (backend) and npm run test:unit (frontend) pass with no skipped tests
- ☐ The Practice 7 story shows a failing-tests commit before its implementation commit in git log
- ☐ No secrets in the repository; .env.example complete in backend/ and frontend/
- ☐ AI_LOG.md, DECISIONS.md and CHECK.md up to date; tag check-7 pushed
- ☐ At the check: a live change task done by hand in 30 minutes, committed and explained

Check 7 · Week 16 (final)

Final check: complete app matching SPEC.md, tests green, README with setup steps, plus a live change task done by hand at the check.

Live change task

30 minutes, CLOSED, by hand, no AI. Each student gets one change of the same size, picked by the teacher from a short list: add an optional field to the main entity end to end (schema, migration, DTO, type, form, list); change a business rule and its error message, such as the loan limit from 3 to 5; add a validation rule with a clear message in the UI; or add a filter to a list endpoint and its page. Allowed: your own code, git and official docs. Commit the change with a clear message, add or update one test where possible, then explain it to the teacher in 2 questions.

How the explain-back works

- **Written (everyone, 25 min, no AI):** three questions pointed at your own tagged repository, for example: open the file that handles login and explain what happens when the password is wrong.
- **Live (about 10 students per check, random, 3 min):** at your screen, explain a piece of code or make a small change by hand while the teacher watches.
- Everyone gets at least 2 live checks during the semester.

How to prepare

After every AI change, read the diff and make sure you could have written it. If you cannot explain it, ask the AI to explain it, then rewrite it in your own way or simplify it.

Before each check, pick the three most important files of the milestone and explain them out loud to yourself.

Academic integrity

- AI use is allowed and expected. It must be logged in AI_LOG.md.
- Copying another student's project, or code you cannot explain, is not allowed.
- Git history matters: one huge commit the night before a check will be questioned at the live explain-back.